



HERRAMIENTAS DE SOFTWARE PARA BIG DATA

HIVE - II

¿QUÉ ES HIVE?



ES UN SOFTWARE QUE FACILITA LA LECTURA, ESCRITURA Y MANEJO DE GRANDES DATASETS RESIDENTES EN ALMACENAMIENTOS DISTRIBUIDOS, UTILIZANDO SQL.

PERMITE PARTICIONES, TABLAS MANAGED/EXTERNAL, LAS CONSULTAS SE TRANSFORMAN EN MAPREDUCE/TEZ

¿QUÉ ES HIVE?



HIVE SOPORTA:

- DATA DEFINITION LANGUAGE (DDL)
- DATA MANIPULATION LANGUAGE (DML)
- USER DEFINED FUNCTIONS (UDF)

CARACTERÍSTICAS



HIVE ES RÁPIDO Y ESCALABLE. PROVEE DE CONSULTAS ESTILO SQL QUE IMPLÍCITAMENTE SE TRANSFORMAN EN MAPREDUCE O SPARK JOBS.

ES CAPAZ DE ANALIZAR GRANDES VOLÚMENES DE DATOS ALMACENADOS EN HDFS.

PERMITE DIFERENTES TIPOS DE ALMACENAMIENTOS COMO TEXTO PLANO, ORC, PARQUET, ETC.

PUEDE TRABAJAR SOBRE DATOS COMPRIMIDOS ALMACENADOS DENTRO DE HDFS.

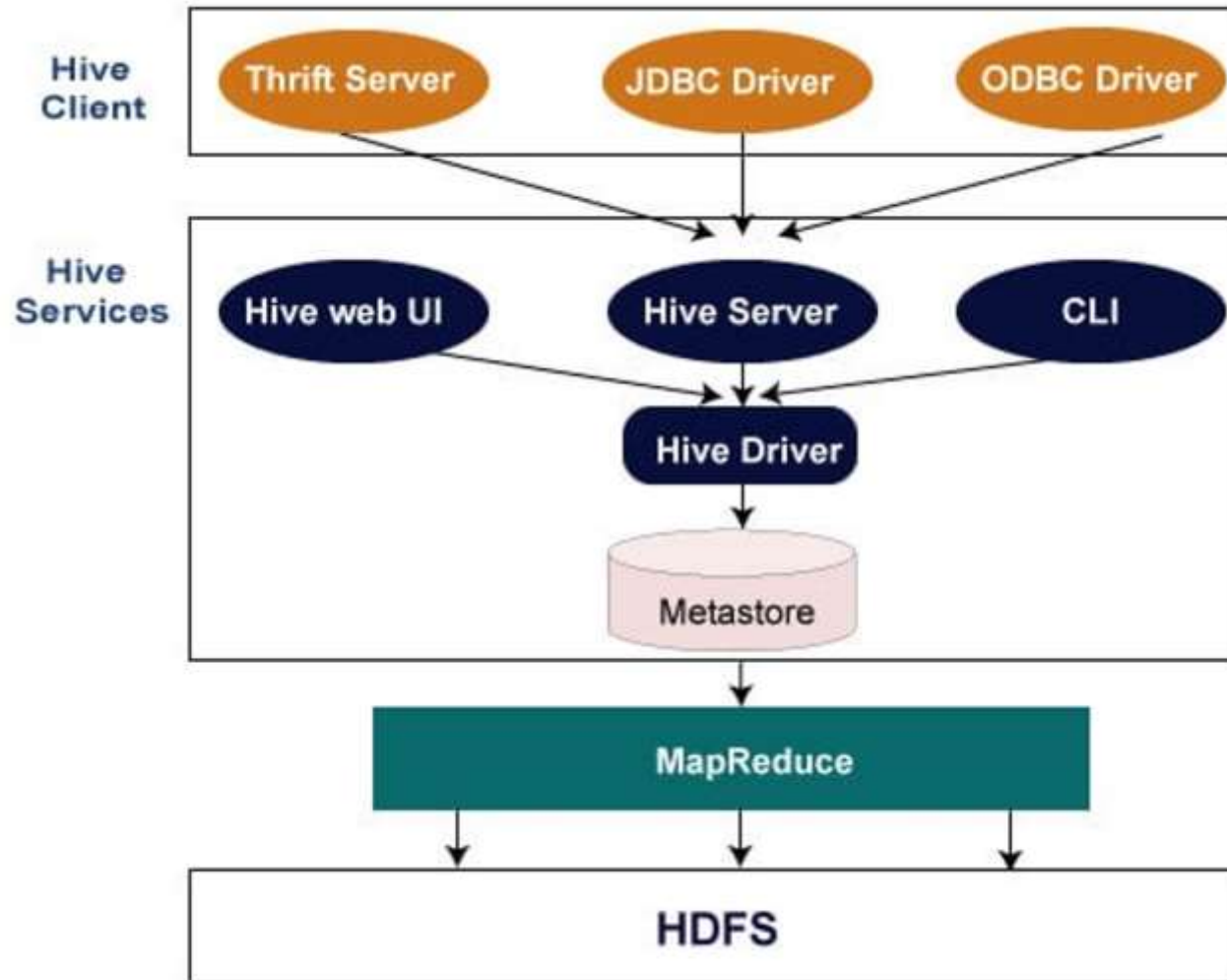
LIMITACIONES



HIVE NO ES CAPAZ DE MANEJAR DATOS EN TIEMPO REAL. SI DE ALMACENOS NO PARA CONSULTARLOS.

HIVE NO ESTÁ DISEÑADO PARA PROCESAMIENTO DE TRANSACCIONES EN LÍNEA.

LAS CONSULTAS HIVE CONTIENEN UNA ALTA LATENCIA.



ARQUITECTURA (HIVE CLIENT)



HIVE PERMITE ESCRIBIR APLICACIONES EN VARIOS LENGUAJES, TALES COMO PYTHON, JAVA, C#, ETC.

POR ESO LA VARIEDAD DE DRIVERS QUE SOPORTA.

ARQUITECTURA (HIVE SERVICES)



HIVE CLI: COMMAND LINE INTERFACE ES UN SHELL DONDE SE PUEDEN EJECUTAR CONSULTAS HIVE Y COMANDOS.

HIVE WEB USER INTERFACE: ES UNA ALTERNATIVA A HIVE CLI, DONDE SE PROVEE DE UNA INTERFAZ WEB PARA REALIZAR LAS MISMAS OPERACIONES.

ARQUITECTURA (HIVE SERVICES)



HIVE METASTORE: REPOSITORIO CENTRAL QUE ALMACENA TODA LA INFORMACIÓN DE LAS TABLAS Y PARTICIONES. TAMBIÉN INCLUYE LA METADATA DE LAS COLUMNAS Y SU INFORMACIÓN DE TIPOS.

HIVE SERVER: REFIERE A APACHE THRIFT SERVER Y ACEPTA EL PEDIDO DE DIFERENTES CLIENTES Y PROVEEDORES AL HIVE DRIVER.

ARQUITECTURA (HIVE SERVICES)



HIVE DRIVER: RECIBE CONSULTAS DESDE DIFERENTES FUENTES TALES COMO UI, CLI, THRIFT, Y JDBC/ODBC DRIVER Y TRANSFIERE LAS CONSULTAS AL COMPILADOR.

HIVE COMPILER: PARSEA LAS CONSULTAS Y REALIZA ANÁLISIS SEMÁNTICO DE LOS DIFERENTES BLOQUES DE LA CONSULTA Y EXPRESIONES. ES EL ENCARGADO DE CONVERTIR EL HIVEQL EN TRABAJOS MAP/REDUCE.

HIVE EXECUTION ENGINE: OPTIMIZA LOS PROCESOS MAPREDUCE Y SUS ACCESOS A HDFS.

TIPOS DE DATOS (NUMÉRICOS)



TINYINT	1 BYTE
SMALLINT	2 BYTE
INT	4 BYTE
BIGINT	8 BYTE
FLOAT	4 BYTE
DOUBLE	8 BYTE

TIPOS DE DATOS (DATETIME)



SOPORTA EL TRADICIONAL UNIX TIMESTAMP.
UN NÚMERO DE TIPO INTEGER, ES INTERPRETADO COMO UNIX
TIMESTAMP EN SEGUNDOS.

UN NÚMERO DEL TIPO FLOAT ES INTERPRETADO COMO UNIX
TIMESTAMP EN SEGUNDOS CON DOBLE PRECISIÓN.

EL RANGO DE LAS FECHAS VA ENTRE: 0000--01--01 A 9999--12--31.

TIPOS DE DATOS (COMPLEJOS)



ARRAYS: CONTIENEN UNA LISTA DE ELEMENTOS DEL MISMO TIPO. ESTOS ELEMENTOS SON ACCEDIDOS POR UN ÍNDICE.

POR EJEMPLO, SI EL CAMPO MASCOTA CONTIENE A LISTA DE ELEMENTOS ['PERRO', 'GATO', 'LORO'], EL ELEMENTO 'GATO' PUEDE SER ACCEDIDO MEDIANTE MASCOTA[1].

TIPOS DE DATOS (COMPLEJOS)



MAPS: CONTIENEN PARES CLAVE-VALOR. CADA ELEMENTO ES ACCEDIDO MEDIANTE SU CLAVE. POR EJEMPLO, UN MAP LISTA_PASSWORDS CONTENIENDO “MAURICIO” COMO CLAVE Y “PASSDEMAURICIO” COMO VALOR, EL PASSWORD DEL USUARIO ES ACCEDIDO MEDIANTE LISTA_PASSWORDS[‘MAURICIO’].

TIPOS DE DATOS (COMPLEJOS)



STRUCTS: CONTIENEN ELEMENTOS DE DIFERENTE TIPOS. CADA ELEMENTO PUEDE SER ACCEDIDO MEDIANTE LA NOTACIÓN PUNTO (DOT NOTATION).

POR EJEMPLO, EN UNA ESTRUCTURA AUTO, EL COLOR DEL AUTO PUEDE SER RECUPERADO MEDIANTE `AUTO.COLOR`.

CREACIÓN BASE DE DATOS



EN HIVE UNA BASE DE DATOS ES CONSIDERADA COMO UN CATÁLOGO O NAMESPACES DE TABLAS.

HIVE PROVEE UNA BASE DE DATOS POR DEFECTO CUYO NOMBRE ES DEFAULT.

PARA LISTAR LAS BASES DE DATOS EXISTENTES SOLO ES NECESARIO HACER:

`SHOW DATABASES;`

PARA CREAR UNA NUEVA BASE DE DATOS SOLAMENTE SE DEBE HACER:

`CREATE DATABASE NOMBREBASEDATOS;`

CREACIÓN BASE DE DATOS



CADA BASE DE DATOS DEBE TENER UN NOMBRE ÚNICO.

SI NO TENEMOS CLARO SI UNA BASE DE DATOS DEL MISMO NOMBRE YA EXISTE PODEMOS HACER: `CREATE A DATABASE IF NOT EXISTS NOMBREBASEDEDATOS.`

HIVE PERMITE AL MOMENTO DE CREAR UNA BASE DE DATOS ASIGNAR PROPIEDADES:

```
CREATE DATABASE NOMBREBASE  
WITH DBPROPERTIES ('CREATOR' = 'USUARIO CREADOR', 'DATE' =  
'2019-06-03');
```

ELIMINACIÓN BASE DE DATOS



PARA ELIMINAR UNA BASE DE DATOS EXISTENTE SOLO SE DEBE HACER:

```
DROP DATABASE NOMBREBASE;
```

SI QUEREMOS ELIMINAR UNA BASE DE DATOS Y NO NOS DE ERROR SI LA MISMA NO EXISTE, PODEMOS HACER:

```
DROP DATABASE IF EXISTS NOMBREBASE;
```

ELIMINACIÓN BASE DE DATOS



EN HIVE NO ESTÁ PERMITIDO ELIMINAR UNA BASE DE DATOS QUE TENGA TABLAS. PARA ELLO, PRIMERO SE DEBEN ELIMINAR LAS TABLAS Y LUEGO LA BASE DE DATOS.

UNA ALTERNATIVA ES EL BORRADO EN CASCADA:
`DROP DATABASE IF EXISTS CURSO CASCADE`

CREACIÓN DE TABLAS



EN HIVE PODEMOS CREAR TABLAS UTILIZANDO CONVENCIONES SIMILARES A SQL COMÚN.

EXISTEN DOS TIPOS DE TABLAS
INTERNAS
EXTERNAS

TABLAS INTERNAS



PARA CREAR UNA TABLA INTERNA DEBEMOS HACER:

```
CREATE TABLE NOMBREBASE.NOMBREALUMNO (ID INT, NAME  
STRING ,EDAD INT)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',' ;
```

ESTE TIPO DE TABLAS SON GENERALMENTE ALMACENADAS EN
UN SUBDIRECTORIO DEBAJO DEL DIRECTORIO DEFINIDO POR EL
HIVE.METASTORE.WAREHOUSE.DIR

(POR EJ /USER/HIVE/WAREHOUSE) EN HDFS

TABLAS INTERNAS



ES IMPORTANTE DESTACAR QUE SI ELIMINAMOS UNA TABLA INTERNA, NO SOLO ELIMINAMOS LA TABLA EN SÍ MISMA, SI NO LOS DATOS ASOCIADOS A LA MISMA, POR ESO HAY QUE TENER MUCHO CUIDADO AL HACERLO.

SI QUEREMOS CREAR UNA TABLA Y ASEGURARNOS DE QUE NO DE ERROR SI YA EXISTE, SOLO LE AGREGAMOS: IF NOT EXISTS.

TABLAS INTERNAS



PARA INSERTAR DATOS DENTRO DE UNA TABLA INTERNA TENEMOS DOS FORMAS:

INSERT INTO TABLE <TABLE_NAME> VALUES (<ADD VALUES AS PER COLUMN ENTITY>);

```
INSERT      INTO      TABLE      STUDENT      VALUES
('DIKSHANT',1,'95'),('AKSHAT', 2 , '96'),('DHRUV',3,'90');
```

TABLAS INTERNAS



HIVE TAMBIÉN NOS DA LA POSIBILIDAD DE CARGAR DATOS A PARTIR DE ARCHIVOS O DATOS PRECARGADOS, YA SEAN LOCALES O EN HDFS, MEDIANTE LOAD DATA.

```
LOAD DATA [LOCAL] INPATH '<THE TABLE DATA LOCATION>'
[OVERWRITE] INTO TABLE <TABLE_NAME>;
```

```
LOAD          DATA          LOCAL          INPATH
'/HOME/DIKSHANT/DOCUMENTS/DATA.CSV'      INTO      TABLE
STUDENT;
```

TABLAS EXTERNAS



LAS TABLAS EXTERNAS NOS PERMITEN CREAR Y ACCEDER A LA TABLA Y LOS DATOS EXTERNAMENTE. EN ESTE TIPO DE TABLAS, LOS DATOS NO SON ALMACENADOS EN EL DIRECTORIO HIVE, POR LO QUE SI ELIMINAMOS UNA TABLA DE ESTE TIPO, SOLO ELIMINAMOS LA TABLA Y NO LOS DATOS ASOCIADOS.

PARA CREAR UNA TABLA EXTERNA PRIMERO QUE NADA DEBEMOS CREAR EL DIRECTORIO HDFS DONDE SE VA A ALMACENAR LA INFORMACIÓN.

TABLAS EXTERNAS



PARA ESO HACEMOS COMO VIMOS EN CLASES ANTERIORES:
HDFS DFS -MKDIR /HIVEDIRECTORY

LUEGO, ALMACENAMOS LA INFORMACIÓN QUE QUERAMOS EN
ESE DIRECTORIO, VÍA LA HERRAMIENTA QUE SE QUIERA POR EJ:

HDFS DFS -PUT HIVE/EMP_DETAILS /HIVEDIRECTORY

TABLAS EXTERNAS



LUEGO, CREAMOS LA TABLA DE LA SIGUIENTE FORMA:

```
CREATE EXTERNAL TABLE CURSO.ALUMNOS (ID INT, NAME  
STRING , CI INT)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/HIVEDIRECTORY';
```

LUEGO PUEDO HACER:

```
SELECT * FROM NOMBREBASE.NOMBRETABLA
```

TABLAS EXTERNAS



LA FORMA DE CARGAR DATOS ES SIMILAR A UNA TABLA INTERNA VÍA `LOAD DATA` O `INSERT INTO`. LA DIFERENCIA RADICA EN DONDE SE ALMACENAN LOS DATOS.

UNA VEZ DEFINIDO EL LUGAR EN HDFS DONDE SE ALMACENAN LOS DATOS, SI SE CARGAN NUEVOS ARCHIVOS A ESE DIRECTORIO LA TABLA VA A TOMAR ESOS DATOS EN FORMA AUTOMÁTICA.

TABLAS EXTERNAS



LOS DATOS DE LAS TABLAS CREADAS EN HIVE PUEDEN SER DE DIFERENTES TIPOS: CSV, ORC, PARQUET, ETC. PARA ELLO AL CREAR LA TABLA SE DEBE ESPECIFICAR EL TIPO (SI NO SE QUIERE EL POR DEFECTO QUE ES ORC).

```
CREATE EXTERNAL TABLE MYTABLE  
(COL1 BIGINT,COL2 BIGINT) ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS ORC  
LOCATION '<ORC FILE LOCATION>'
```

ELIMINAR TABLAS



HIVE OFRECE LA POSIBILIDAD DE ELIMINAR TABLAS ANTERIORMENTE CREADAS (YA SEAN INTERNAS O EXTERNAS) MEDIANTE EL COMANDO DROP.

`DROP TABLE NOMBREBASE.NOMBRETABLA;`

RECUERDEN QUE SI ELIMINAN UNA TABLA INTERNA, LA DATA CONTENIDA EN LA MISMA ES ELIMINADA TAMBIÉN, MIENTRAS QUE EN UNA TABLA EXTERNA, LA DATA CONTINÚA SIN PROBLEMAS.

MODIFICAR TABLAS



EN HIVE PODEMOS HACER MODIFICACIONES, ACTUALIZACIONES EN TABLAS EXISTENTES COMO CAMBIAR EL NOMBRE DE LA TABLA, EL NOMBRE DE ALGUNA COLUMNA, AGREGAR PROPIEDADES, ETC.

RENOMBRAR



SI QUEREMOS CAMBIAR EL NOMBRE DE UNA TABLA HACEMOS:

```
ALTER TABLE OLD_TABLE_NAME RENAME TO NEW_TABLE_NAME;
```

ELIMINAR COLUMNAS



HIVE NOS DA LA POSIBILIDAD DE ELIMINAR COLUMNAS DE TABLAS YA EXISTENTES VÍA REPLACE. SE TIENE ESTA TABLA DE NOMBRE DATOS QUE TIENE: ID (INT), NAME (STRING), SALARY(FLOAT), AGE(INT) Y SE HACE:

```
ALTER TABLE DATOS REPLACE COLUMNS( ID STRING, FIRST_NAME  
STRING, AGE INT);
```

LA NUEVA TABLA TENDRÁ:
ID (INT), FIRST_NAME (STRING), AGE(INT)

PARTICIONAMIENTO



EL PARTICIONAMIENTO EN HIVE SIGNIFICA DIVIDIR LOS DATOS CONTENIDOS EN UNA TABLA EN VARIAS PARTES DE ACUERDO A UNA COLUMNA PARTICULAR.

PERO, ¿POR QUÉ ES ÚTIL ESTO?

PARTICIONAMIENTO



HADOOP ES UTILIZADO PARA MANEJAR GRANDES VOLÚMENES DE DATOS, POR LO QUE SIEMPRE REQUIERE LA MEJOR MANERA DE TRABAJAR CON LOS MISMOS. EL PARTICIONAMIENTO ES UNA DE ELLAS.

SUPONGAMOS QUE TENGAMOS UNA TABLA CON 10 MILLONES DE REGISTROS QUE CONTIENE DATOS DE ESTUDIANTES Y QUEREMOS SABER QUÉ ESTUDIANTES CURSAN UN CURSO DETERMINADO.

SI CONTAMOS CON UNA COLUMNA CURSO Y PARTICIONAMOS POR ELLA, CUANDO VAMOS A BUSCAR DETERMINADO CURSO, IRÍAMOS A ESA INFORMACIÓN.

PARTICIONAMIENTO



EXISTEN DOS TIPOS DE PARTICIONAMIENTO:

ESTÁTICO Y DINÁMICO

PARTICIONAMIENTO ESTÁTICO



SE CONFIGURA AL MOMENTO DE DEFINIR LA TABLA Y ES REQUERIDO PASAR LOS VALORES DE LAS COLUMNAS PARTICIONADAS MIENTRAS SE CARGAN LOS DATOS A LA TABLA.

```
CREATE TABLE STUDENT (ID INT, NAME STRING, AGE INT,  
INSTITUTE STRING)  
PARTITIONED BY (COURSE STRING)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ';;'
```

```
INSERT OVERWRITE TABLE STUDENT PARTITION (COURSE='MATH')  
SELECT COL1, COL2, COL2, COL4 FROM FUENTE;
```

PARTICIONAMIENTO DINÁMICO



EN EL CASO DEL PARTICIONAMIENTO DINÁMICO, NO ES NECESARIO PASAR LOS VALORES DE LAS COLUMNAS PARTICIONADAS. CREAMOS UNA TABLA:

```
CREATE TABLE STUD_DEMO(ID INT, NAME STRING, AGE INT,  
INSTITUTE STRING, COURSE STRING)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ';;'
```

LUEGO, PERMITIMOS EL PARTICIONAMIENTO DINÁMICO VIA:

```
HIVE> SET HIVE.EXEC.DYNAMIC.PARTITION=TRUE;  
HIVE> SET HIVE.EXEC.DYNAMIC.PARTITION.MODE=NONSTRICT;
```

PARTICIONAMIENTO DINÁMICO



LA TABLA HASTA AHORA NO TIENE PARTICIÓN PERO SI HACEMOS UNA TABLA DUMMY DEL TIPO:
CREATE TABLE STUDENT_PART (ID INT, NAME STRING, AGE INT, INSTITUTE STRING)
PARTITIONED BY (COURSE STRING)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

PARTICIONAMIENTO DINÁMICO



LUEGO INSERTAMOS DATOS:

```
INSERT INTO STUDENT_PART  
PARTITION(COURSE)  
SELECT ID, NAME, AGE, INSTITUTE, COURSE  
FROM STUD_DEMO;
```

ESTO NOS PERMITE IR CREANDO NUEVAS PARTICIONES POR COLUMNAS A MEDIDA INSERTAMOS DATOS.

OPERADORES ARITMÉTICOS



$A + B$ SUMA COLUMNA $A + B$

$A - B$ RESTA COLUMNA $A - B$

$A * B$ MULTIPLICA COLUMNA $A * B$

A / B DIVIDE A SOBRE B

$A \% B$ RETORNA EL RESTO DE A SOBRE B

OPERADORES RELACIONALES



$A=B$ TRUE SI $A = B$ FALSE EN CASO CONTRARIO

$A <> B$, $A \neq B$ NULL SI A O B SON NULOS. TRUE SI $A <> B$ FALSE EN CASO CONTRARIO

$A < B$ NULL SI A O B SON NULOS. TRUE SI A MENOR QUE B FALSE EN CASO CONTRARIO

FUNCIONES MATEMÁTICAS



ROUND(NUM) REDONDEA EL NÚMERO DADO.

FLOOR(NUM) RETORNA EL ENTERO MENOR O IGUAL AL NÚMERO.

EXP(NUM) RETORNA EL EXPONENCIAL DEL NÚMERO.

LN(NUM) RETORNA EL LOGARTIMO NATURAL DEL NÚMERO.

FUNCIONES MATEMÁTICAS



LOG10(NUM) RETORNA EL LOGARITMO BASE 10 DEL NÚMERO.

SQRT(NUM) RAÍZ CUADRADA DEL NÚMERO.

ABS(NUM) VALOR ABSOLUTO DEL NÚMERO

FUNCIONES BUILT-IN



LENGTH(STR) LARGO DEL STRING

REVERSE(STR) RETORNA EL STRING EN FORMA INVERSA.

CONCAT(STR1, STR2, ...) CONCATENA LOS STRINGS.

SUBSTR(STR, START_INDEX) RETORNA EL SUBSTRING DE STR COMENZANDO EN START_INDEX.

FUNCIONES BUILT-IN



LENGTH(STR) LARGO DEL STRING

REVERSE(STR) RETORNA EL STRING EN FORMA INVERSA.

CONCAT(STR1, STR2, ...) CONCATENA LOS STRINGS.

SUBSTR(STR, START_INDEX) RETORNA EL SUBSTRING DE STR COMENZANDO EN START_INDEX.

FUNCIONES BUILT-IN



SUBSTR(STR, INT START, INT LENGTH) RETORNA EL SUBSTRING DE STR COMENZANDO EN START Y DE LARGO LENGTH.

SQRT(NUM) RAÍZ CUADRADA DEL NÚMERO..

ABS(NUM) VALOR ABSOLUTO DEL NÚMERO.

GOBIERNO DE DATOS



AL IGUAL DE LO QUE HABLAMOS CON HDFS EL GOBIERNO ES FUNDAMENTAL DENTRO DE HIVE. BÁSICAMENTE DEBERÍAMOS TENER ESQUEMAS PARA DESARROLLO Y ESQUEMAS PARA DEJAR EN PRODUCCIÓN.

ORGANIZAR LOS DATOS EN MODELOS EN DISTINTAS TABLAS COLABORA CON EL GOBIERNO DE DATOS, EL CATALOGADO DE LOS MISMOS, LA TRAZABILIDAD Y LA INTEGRIDAD. LOGRAMOS UNA ÚNICA REALIDAD CON LOS DATOS QUE ES TRANSVERSAL A LA ORGANIZACIÓN.

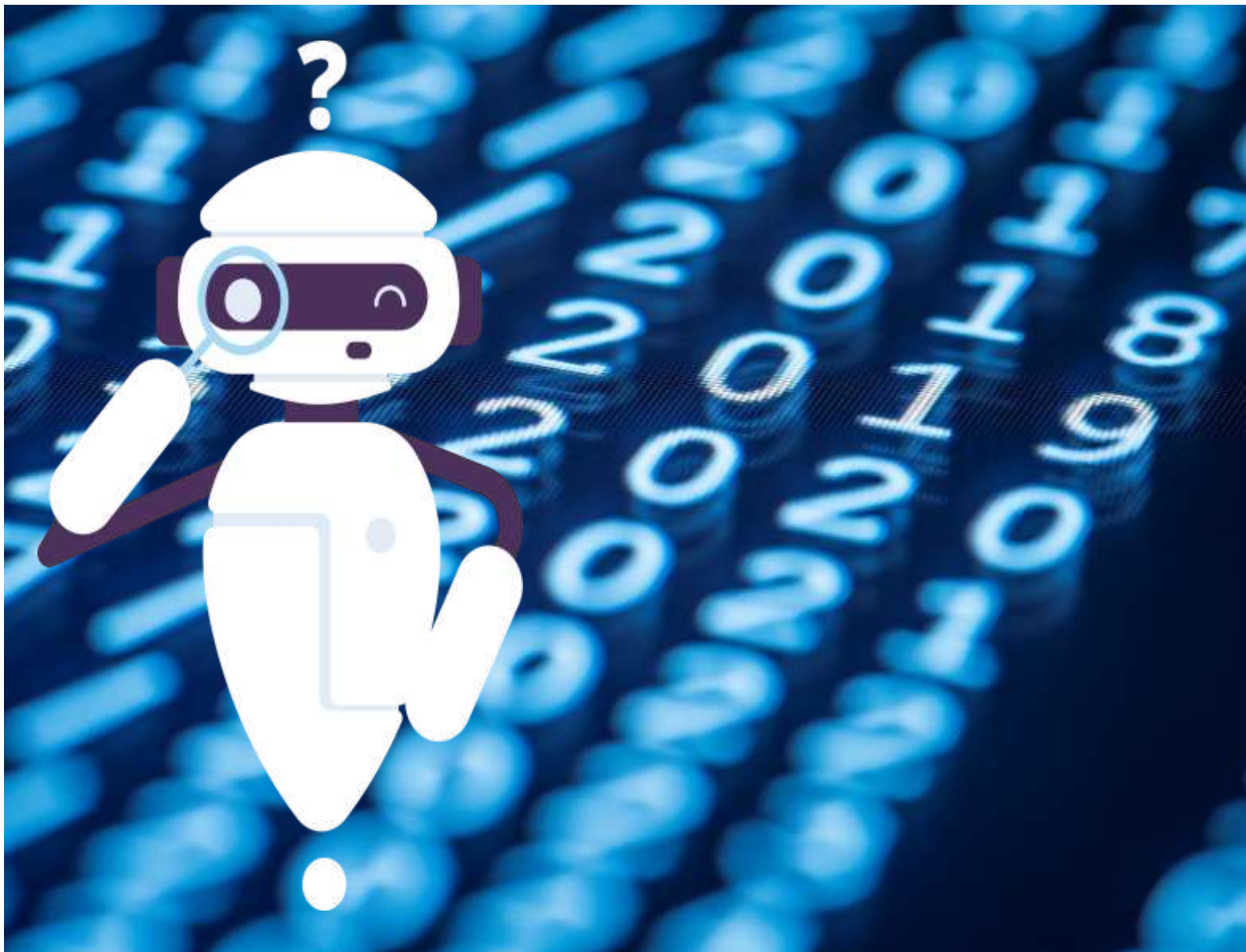
GOBIERNO DE DATOS



ES RECOMENDABLE QUE NO TODOS TENGAN PERMISO PARA CREAR TABLAS O ACCEDER A LOS DATOS QUE HAY EN ELLAS. HAY DIFERENTES HERRAMIENTAS COMO POR EJEMPLO APACHE RANGER QUE NOS PERMITEN GESTIONAR LOS PERMISOS PARA CREAR TABLAS TANTO EN ESQUEMAS DE DESARROLLO COMO DE PRODUCCIÓN, ASÍ COMO PARA ACCEDER A LOS MISMOS. RANGER TAMBIÉN CONTROLA EL ACCESO A TODOS LOS DIRECTORIOS QUE CREAMOS EN HDFS.

POSIBLES PREGUNTAS DE PARCIAL

- Explicar qué es Apache Hive y cuál es su función dentro de un entorno Big Data. ¿Por qué es útil en el análisis de grandes volúmenes de datos?
- Describir brevemente las principales características de Hive. ¿Qué lo hace escalable y apto para trabajar con datos almacenados en HDFS?
- ¿Cuáles son las limitaciones de Hive? Mencionar al menos dos y explicar por qué representan una desventaja en ciertos escenarios.
- Comparar las tablas internas y externas en Hive. ¿Qué diferencias existen en cuanto al almacenamiento y la eliminación de datos?
- Explicar qué es el Hive Metastore y qué tipo de información almacena. ¿Por qué es un componente esencial en la arquitectura de Hive?
- ¿Qué es el particionamiento en Hive? ¿Cuál es su ventaja principal y en qué se diferencian el particionamiento estático y dinámico?
- Describir qué son los tipos de datos complejos en Hive con ejemplos breves: ARRAY, MAP y STRUCT.
- Explicar el proceso general para crear una base de datos en Hive, incluyendo cómo prevenir errores si la base ya existe y cómo asignar propiedades a la misma.
- Mencionar y explicar dos funciones matemáticas y dos funciones de texto (built-in) disponibles en Hive. Incluir un ejemplo de uso de cada una.
- ¿Qué medidas de gobierno de datos se recomienda adoptar al utilizar Hive? ¿Qué herramientas se pueden utilizar para gestionar los permisos sobre los datos?



GRACIAS

¿ALGÚN PUNTO QUE
QUIERAN REPASAR O
PROFUNDIZAR?